Java Multithreading Examples

Introduction to Multithreading

Multithreading is a core concept in Java that allows concurrent execution of two or more threads.
Each thread runs independently, sharing the same memory space. It improves application performance,
especially in tasks that are I/O or CPU-intensive.

Benefits:
- Efficient use of CPU resources.
- Better application responsiveness.
- Useful in performing background tasks like logging, file I/O, or network operations.

Ways to Create Threads in Java:
1. By extending the Thread class
2. By implementing the Runnable interface
3. Using ExecutorService (preferred for real applications)

1. Extending Thread Class

Example 1: By Extending the Thread Class

```
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
                        System.out.println("From  MyThread  -  Count:  "  +  i  +  "  -  "  +
Thread.currentThread().getName());
        }
    }
}

public class ThreadExample1 {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        thread1.start();
        thread2.start();

        System.out.println("Main Thread: " + Thread.currentThread().getName());
    }
}
```

2. Implementing Runnable Interface

Example 2: By Implementing the Runnable Interface

```
class MyRunnableTask implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
                    System.out.println("From MyRunnableTask - Count: " + i + " - " +
Thread.currentThread().getName());
        }
    }
}

public class ThreadExample2 {
    public static void main(String[] args) {
        Thread thread1 = new Thread(new MyRunnableTask());
        Thread thread2 = new Thread(new MyRunnableTask());

        thread1.start();
        thread2.start();

        System.out.println("Main Thread: " + Thread.currentThread().getName());
    }
}
```

3. Using ExecutorService

```
Example 3: Using ExecutorService

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class MyWorkerTask implements Runnable {
    private final String taskName;

    public MyWorkerTask(String taskName) {
        this.taskName = taskName;
    }

    @Override
    public void run() {
        for (int i = 1; i <= 2; i++) {
                        System.out.println(taskName + " - Count: " + i + " - " +
Thread.currentThread().getName());
        }
    }
}

public class ThreadExample3 {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3); // Pool of 3 threads
```

```
        executor.execute(new MyWorkerTask("Task-1"));
        executor.execute(new MyWorkerTask("Task-2"));
        executor.execute(new MyWorkerTask("Task-3"));
        executor.execute(new MyWorkerTask("Task-4"));

        executor.shutdown(); // Always shut down the executor
        System.out.println("Main Thread: " + Thread.currentThread().getName());
    }
}
```

Summary

Summary:

Approach	Flexibility	Reusability	Real-world Use
extends Thread	Less	No	Rare
implements Runnable	Better	Yes	Common
ExecutorService	Best	Yes	Preferred